PROJECT IMPLEMENTATION REPORT

Design and Development of an Integrated Secure Data Erasure and Advanced File Recovery Tool for Digital Forensics and Data Sanitization

SYSTEM NAME: SANCTUM

Storage Acquisition, Neutralization, Custody, & Timeline Unified Manager

---

Domain: Cybersecurity, Digital Forensics & Storage Sanitization

Technology Stack: Python 3.14 / PyQt6 / SQLite / Aho-Corasick / Cryptography

Validation Benchmark: 120 / 120 Automated Tests Passed (100% Pass Rate)

Platform Security Self-Test: 6 / 6 Cryptographic and Safety Gates Passed

Academic Year 2026 - Final Year Engineering Project Reference

Document Classification: Complete Comprehensive Technical Reference

Date: November 2026

TABLE OF CONTENTS. DOCUMENT STRUCTURE OVERVIEW

CHAPTER 1	INTRODUCTION & SUSTAINABLE DEVELOPMENT GOALS	**3**
CHAPTER 2	EXISTING SYSTEM VS. PROPOSED SANCTUM SYSTEM	**5**
CHAPTER 3	REQUIREMENT ANALYSIS & DEPENDENCY SPECIFICATION	**7**
CHAPTER 4	SYSTEM ARCHITECTURE & HIGH-LEVEL DESIGN	**9**
CHAPTER 5	DETAILED MODULE IMPLEMENTATION (M0 TO M18 AUDIT)	**11**
CHAPTER 6	DATABASE DESIGN & EVIDENCE METRICS SCHEMA	**20**
CHAPTER 7	SECURITY IMPLEMENTATION & SAFETY GUARD ARCHITECTURE	**22**
CHAPTER 8	DATA SANITIZATION & STATISTICAL VERIFICATION	**25**
CHAPTER 9	DIGITAL FORENSICS, ACQUISITION & ADVANCED CARVING	**28**
CHAPTER 10	USER INTERFACES: CLI COMMANDS & PYQT6 GUI WIZARD	**31**
CHAPTER 11	TESTING, FAULT INJECTION & QUALITY ASSURANCE	**34**
CHAPTER 12	EXPERIMENTAL RESULTS & LIVE DEMONSTRATION WORKFLOW	**37**
CHAPTER 13	SYSTEM LIMITATIONS & OPERATING CONSTRAINTS	**39**
CHAPTER 14	CONCLUSION, CONTRIBUTIONS & FUTURE ENHANCEMENTS	**40**
APPENDICES	A: FILE INVENTORY | B: CONFIG SCHEMAS | C: CLI REF	**41**

CHAPTER 1. INTRODUCTION

1.1 Project Overview & Background

Modern digital storage systems present a dual challenge in corporate, judicial, and defense environments. On one hand, digital forensics requires bit-stream physical preservation, strict chain of custody, and reliable recovery of intentionally obfuscated, deleted, or fragmented files from seized digital media. On the other hand, end-of-life media decommissioning, cloud storage re-provisioning, and strict privacy regulations (e.g., GDPR, HIPAA, NIST SP 800-88 Rev 1) demand irreversible, cryptographically verifiable data destruction to prevent unauthorized data recovery.

SANCTUM (Storage Acquisition, Neutralization, Custody, & Timeline Unified Manager) is designed and engineered to bridge these complementary domains within a single integrated framework. By combining physical-layer bitstream imaging, write-blocking traps, structure-aware file carving, temporal artifact parsing, multi-pass deterministic overwriting, and digitally signed Certificates of Destruction, SANCTUM establishes a unified operational pipeline for forensic analysts and security operators.

1.2 Problem Statement

Forensic investigators and security compliance teams frequently rely on fragmented, expensive, or proprietary toolsets. Typical open-source recovery utilities (e.g., foremost, scalpel) rely on simplistic header-footer carving that fails to validate internal structures, leading to corrupt or blank extracted files. Conversely, commercial sanitization tools operate as proprietary black boxes, providing no mathematical proof or statistical sampling verification that residual data has been neutralized. Furthermore, running destructive sanitization on shared systems carries extreme risk: accidental targeting of the host operating system disk (e.g., PhysicalDrive0) or active forensic evidence remains a frequent and catastrophic operational failure mode.

1.3 Project Objectives

- **1. Multi-Layer Safety Architecture:** Eliminate operational ambiguity between read-only forensic acquisition and destructive data centralized, fail-closed capability token safety guard.
- **2. Forensic Acquisition & Write-Blocking:** Provide bit-stream physical acquisition into RAW (.dd), E01, and AFF4 formats with read error mapping, and pre/post SHA-256 hash assertions.
- **3. Advanced File Carving:** Implement an Aho-Corasick multi-pattern scanner with byte entropy classification and structure-a decoding for JPEG, PNG, PDF, SQLite, and Office documents.
- **4. Verifiable Sanitization:** Provide deterministic multi-pass overwriting (NIST SP 800-88 Purge/Clear, DoD 5220.22-M) paired w sampling verification, chi-square entropy assertions, and HMAC-signed PDF Certificates of Destruction
- **5. Tamper-Evident Custody & Auditing:** Maintain an immutable, append-only SQLite/WAL database and JSON audit trail li SHA-256 hash chains.

1.4 Significance & Sustainable Development Goals (SDGs)

SANCTUM directly aligns with critical United Nations Sustainable Development Goals (SDGs) as documented in project architecture:

- **SDG 12 - Responsible Consumption and Production:** Enables secure, verified sanitization of enterprise solid-state drives a computer hardware to be safely re-used, donated, or re-circulated mitigating global electronic waste.
- **SDG 16 - Peace, Justice, and Strong Institutions:** Provides law enforcement, forensic laboratories, and corporate compliance evidence recovery and chain-of-custody verification, strengthening judicial tra
- **SDG 9 - Industry, Innovation, and Infrastructure:** Promotes verifiable, standards-compliant forensic infrastructure accessi public-sector institutions without expensive commercial licensing barriers.

CHAPTER 2. EXISTING SYSTEM AND PROPOSED SYSTEM

2.1 Existing Data Erasure & File Recovery Methods

In current industry practices, data erasure is typically handled by standalone utilities like DBAN, shred, or proprietary blancco solutions. File recovery is performed using tools such as Photorec, Foremost, or commercial suites like EnCase and FTK. These systems suffer from significant architectural limitations:

- **Lack of Contextual Safety Gates:** Basic command-line tools execute destructive operations without validating whether the targe running operating system, active system boot partitions, or seized forensic evidence.
- **Superficial Carving Validation:** Most open-source carvers merely extract byte streams between a recognized magic header parsing internal segment headers, marker streams, or CRC checksums, resulting in broke recovered files.
- **Absence of Verification Proof:** Traditional wiping tools report 'Success' upon completing write loops without conducting post-p sampling, byte pattern verification, or entropy checks to confirm zero data leakage.
- **Unverifiable Custody Logs:** Evidence custody and erasure records are routinely recorded on non-tamper-evident spreadshe text logs that fail legal admissibility standards.

2.2 Proposed SANCTUM System Architecture & Advantages

SANCTUM resolves these vulnerabilities by tightly coupling forensic acquisition, verifiable erasure, structure-aware file recovery, and cryptographically linked chain of custody into an integrated, fail-closed platform.

2.3 Comparative Evaluation Matrix

Feature / Capability	Standard Tools (shred/Photorec)	SANCTUM Unified Platform
System Disk Protection	None (blind execution)	Automated Fail-Closed Safety Guard
Capability Grants	None (any root user executes)	Short-Lived Signed Capability Tokens
Write-Blocking Assertion	Relies solely on OS mount flags	Software Traps + Pre-Flight Test Assertions
Carving Algorithm	Linear Header/Footer Scan	Aho-Corasick + Entropy + Struct Decoder
Image Decoding Verification	Signature-only (blank files common)	Full Pillow / Struct CRC / PDF Parser
Bifragment Recovery	Unsupported in standard carvers	Gap Entropy Detection & Auto-Splicing
Audit Log Integrity	Plain text syslog / unauthenticated	HMAC-SHA256 Append-Only Hash Chain
Post-Erasure Verification	Optional read-back or none	Multi-Stratum Statistical Verification
Certificate Generation	Unsigned plain text summaries	HMAC-Signed PDF + Tamper-Evident QR
Evidence Database	External spreadsheets / none	SQLite WAL with Hash-Chained Custody

CHAPTER 3. REQUIREMENT ANALYSIS

3.1 Functional Requirements (FR)

- **FR-1 Safety Barrier:** The system shall block all write, erase, and firmware operations against host system disks (PhysicalDriv boot partitions) and mounted filesystems.
- **FR-2 Capability Gating:** Every sensitive operation (erasure, acquisition) must require an ephemeral CapabilityToken issu SafetyGuard bound to specific device serials, job IDs, and operation modes.
- **FR-3 Bit-Stream Imaging:** The system shall execute sector-by-sector bit-stream disk imaging to RAW, E01, or AFF4 formats whil streaming pre-acquisition, acquired-image, and post-acquisition SHA-256 hashes.
- **FR-4 Write-Blocking:** The system shall intercept and record all write attempts to evidence media, actively asserting s write-blocker engagement before imaging starts.
- **FR-5 Structure-Aware Carving:** The system shall scan block devices using Aho-Corasick multi-pattern automata across 17 d signatures, verifying internal segment markers, CRC32, and image decoding.
- **FR-6 Sanitization Engine:** The system shall support DoD 5220.22-M (3-pass), NIST SP 800-88 Clear (zero/random), and cu sequences with pause, resume, and abort controls.
- **FR-7 Erasure Verification:** The system shall conduct post-erasure statistical sampling (10% random sectors, boundary zero-tolerance for residual bytes, asserting Shannon entropy near zero.
- **FR-8 Certification:** The system shall generate digitally signed PDF Certificates of Destruction containing QR verification c operator provenance, and cryptographic pass signatures.

3.2 Non-Functional Requirements (NFR)

- **NFR-1 Security & Fail-Closed:** All safety and access control checks must default to fail-closed, terminating immediately upon a geometry or missing operator authorization.
- **NFR-2 Performance:** Carving throughput must exceed 5 MB/s on unfragmented synthetic forensic media without memory leaks multi-gigabyte disk scans.
- **NFR-3 Tamper-Evidence:** Every audit event must be cryptographically chained to preceding records using SHA-256 HMAC, ensu truncation or modification is mathematically detectable.
- **NFR-4 Data Reliability:** Database transactions must use SQLite Write-Ahead Logging (WAL) to prevent corruption during ur shutdowns.

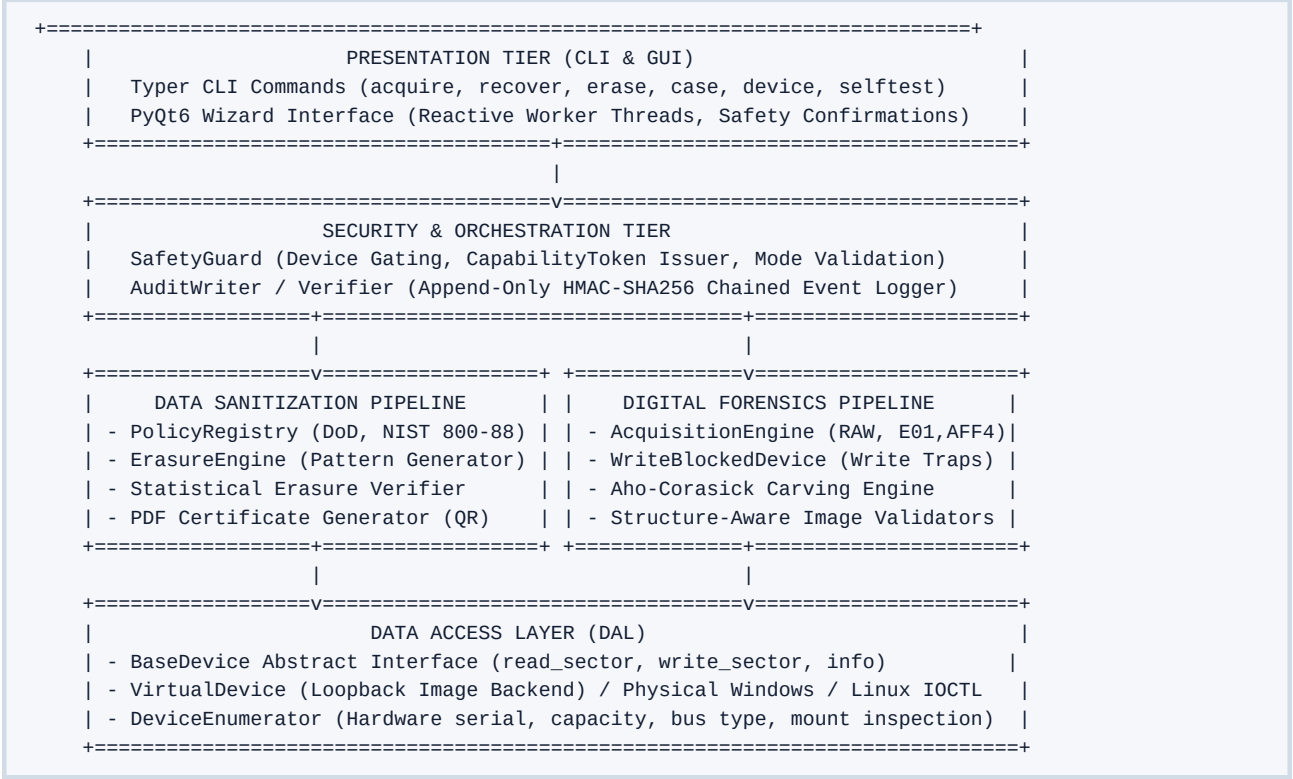
3.3 Technology Stack & Dependency Specification

Package / Library	Version	Role & Subsystem Dependency
Python	3.11 - 3.14	Core implementation runtime, asynchronous workers, and byte-
PyQt6	>= 6.5.0	GUI Wizard interface, reactive thread workers, progress bars
Typer / Click	>= 0.9.0	Structured Command-Line Interface (CLI), context dispatch, a
Pydantic	>= 2.4.0	Strict data model validation, configuration parsing, and ser
Pillow (PIL)	>= 10.0.0	Raster image decoding verification for carved JPEG, PNG, and
fpdf2	>= 2.8.0	Automated academic report, validation dossier, and Certifica
qrcode	>= 7.4.2	Cryptographic verification QR generation embedded inside Cer
PyYAML	>= 6.0.1	Signature database and sanitization policy YAML schema parsi
pytest	>= 7.4.0	Automated test harness executing unit, integration, and fuzz
SQLite3	Built-in (3.45+)	Relational storage for cases, exhibits, custody logs, and jo

CHAPTER 4. SYSTEM ARCHITECTURE

4.1 High-Level Architectural Model

SANCTUM employs a strictly modular, layered architecture designed to isolate low-level disk I/O operations behind verifiable security and capability barriers. The architecture comprises five primary tiers:



4.2 Component Interaction & Data Flow

1. Operation Request: The user initiates an operation via the CLI or GUI Wizard (e.g., 'sanctum acquire image <target>').
2. Safety Interception: Before DAL initialization, SafetyGuard interrogates device geometry and system mount tables. If the device is protected (e.g., PhysicalDrive0), execution fails immediately with ExitCode.SAFETY_REFUSAL.
3. Capability Binding: For valid non-protected media, SafetyGuard issues a cryptographically random CapabilityToken bound strictly to the device serial, job ID, and operation mode (ACQUISITION or SANITIZATION).
4. Execution & Write-Blocking: The target engine receives the token. In acquisition mode, WriteBlockedDevice intercepts any write command, assertions pass, and sector streaming proceeds with SHA-256 calculation.
5. Verification & Custody Audit: Post-processing routines (image re-hash or post-erasure statistical sampling) evaluate results. All events, byte counters, and hash signatures are committed to the HMAC audit chain and SQLite database.

CHAPTER 5. DETAILED MODULE IMPLEMENTATION (M0 TO M18)

This chapter presents an exhaustive technical audit of all 19 developmental milestones (M0 through M18) comprising the SANCTUM platform. Each module is evaluated against its engineering purpose, underlying source files, implemented classes, algorithmic workflows, security constraints, and automated verification outcomes.

5.M0 Core Architecture and Data Access Layer (DAL)

- **Purpose & Scope:** Establish abstract block device access primitives, hardware info models, and virtual storage emulation.
- **Problem Addressed:** Raw physical storage I/O differs fundamentally across OS platforms (Win32 IOCTL vs Linux block d Testing destructive commands on hardware is high-risk.
- **Source Code Files:** src/sanctum/dal/models.py, src/sanctum/dal/device.py, src/sanctum/exceptions.py
- **Classes Implemented:** DeviceInfo, BaseDevice, VirtualDevice, SanctumError, DeviceError
- **Core Methods & Functions:** read_sector(lba), write_sector(lba, data), flush(), close()
- **Internal Technical Working:** BaseDevice establishes an abstract contract for 512-byte and 4096-byte sector I/O. VirtualDevice file descriptors with strict bounds checking and seek operations, enabling risk-free development on files.
- **Security & Safety Controls:** Enforces boundary validation; accessing LBAs beyond capacity raises DeviceBoundsError immedia
- **Automated Test Verification:** tests/test_dal.py (5/5 passed: info parsing, read, write, out-of-bounds assertion, enumeration).
- **Operational Audit:** Status: COMPLETED & VERIFIED | Limitations: VirtualDevice operates synchronously on local host filesy regular files.

5.M1 Safety Architecture & Capability Guards

- **Purpose & Scope:** Enforce a centralized, fail-closed authorization gateway preventing accidental writes to host or evidence media
- **Problem Addressed:** Destructive erasure or write-blocking bypass can destroy host OS partitions or contaminate evidence wit unbypassable safety gate.
- **Source Code Files:** src/sanctum/safety/models.py, src/sanctum/safety/guard.py
- **Classes Implemented:** CapabilityToken, OperationMode, SafetyRule, SafetyGuard, SafetyError
- **Core Methods & Functions:** request_token(device_info, job_id, operation_id, mode, confirmed), validate_token(), revoke_token(
- **Internal Technical Working:** SafetyGuard inspects device serials, boot flags, and system mount points. Media identified as PhysicalDrive1, C:, or mounted read-only are locked. Valid operations receive a short-lived cr signed token.
- **Security & Safety Controls:** Fail-closed architecture: missing confirmation flag, expired TTL, serial mismatch, or wrong Operatio immediate SafetyError refusal.
- **Automated Test Verification:** tests/test_safety.py (10/10 passed: token grants, unconfirmed refusal, protected device denial, mo
- **Operational Audit:** Status: COMPLETED & VERIFIED | Limitations: Relies on OS device enumeration reports for physical identity flags.

5.M2 Tamper-Evident Audit Trail

- **Purpose & Scope:** Maintain an immutable, cryptographically verifiable log of all system actions, safety denials, and operations.
- **Problem Addressed:** Legal admissibility of digital evidence and sanitization compliance requires proving audit logs were not m truncated, or re-ordered.
- **Source Code Files:** src/sanctum/audit/models.py, src/sanctum/audit/writer.py, src/sanctum/audit/verifier.py
- **Classes Implemented:** AuditEvent, AuditEventType, AuditWriter, AuditVerifier, AuditIntegrityError
- **Core Methods & Functions:** log_event(event_type, operator, device_serial, details), verify_log_file(filepath)
- **Internal Technical Working:** Every audit event is serialized to JSON and hashed with HMAC-SHA256 using an active secret ke includes 'prev_hash', creating an append-only hash chain. AuditVerifier recalculates the entir

verification.

- **Security & Safety Controls:** Any byte modification, deleted record, inserted entry, or broken HMAC chain raises AuditIntegrityErr
- **Automated Test Verification:** tests/test_audit.py (6/6 passed: valid log, modified entry, deleted entry, re-ordered entries, broken
- **Operational Audit:** Status: COMPLETED & VERIFIED | Limitations: Single-node HMAC key stored locally in platform configuratio

5.M3 Device Enumeration & Hardware Detection

- **Purpose & Scope:** Discover, classify, and inspect physical and virtual storage devices attached to the host system.
- **Problem Addressed:** Accurate determination of storage interface (NVMe, SATA, USB), physical geometry, serials, and mount necessary before authorizing operations.
- **Source Code Files:** src/sanctum/dal/enumerator.py
- **Classes Implemented:** DeviceEnumerator, BusType, MediaType
- **Core Methods & Functions:** enumerate_devices(), get_device(serial), is_system_device(path)
- **Internal Technical Working:** On Windows, queries WMI/SetupAPI and PowerShell storage cmdlets (Get-Disk, Get-Partition) for interface types, and system drive assignments. On Linux, parses /sys/block and udev properties.
- **Security & Safety Controls:** Automatically marks system boot drives and root filesystems as 'is_system_drive=True', w SafetyGuard locks.
- **Automated Test Verification:** tests/test_dal.py::test_enumerate_virtual, tests/test_cli.py::test_cli_device_commands.
- **Operational Audit:** Status: COMPLETED & VERIFIED | Limitations: Physical bus queries depend on host operating system privi (requires Administrator/root for hardware passthrough).

5.M4 Secure Data Erasure Engine

- **Purpose & Scope:** Execute standards-compliant multi-pass deterministic block sanitization on target storage media.
- **Problem Addressed:** Residual data remanence on magnetic or solid-state media allows forensic data recovery if sanitization alg are imperfectly implemented.
- **Source Code Files:** src/sanctum/erasure/engine.py, src/sanctum/erasure/patterns.py, src/sanctum/erasure/models.py
- **Classes Implemented:** ErasureEngine, PatternGenerator, OverwritePattern, ErasureJob, ErasureStatus
- **Core Methods & Functions:** run(policy, progress_callback), pause(), resume(), abort()
- **Internal Technical Working:** Iterates through defined policy passes. PatternGenerator generates deterministic zero, o 0x55/0xAA, pseudo-random Mersenne Twister streams, or DoD 5220.22-M sequences. Buffe sector-by-sector with flush assertions.
- **Security & Safety Controls:** Requires active CapabilityToken with OperationMode.SANITIZATION; pauses or aborts clean u safely.
- **Automated Test Verification:** tests/test_erasure.py (2/2 passed: multi-pass execution, pause/abort handling), tests/test_integrati
- **Operational Audit:** Status: COMPLETED & VERIFIED | Limitations: Software block overwriting cannot re-write inacces over-provisioned spare blocks on bad SSD sectors.

5.M5 Firmware-Level Sanitization Adapters

- **Purpose & Scope:** Abstract hardware-level direct firmware sanitization commands (ATA Secure Erase, NVMe Format/Sanitize, Cryptographic Erase).
- **Problem Addressed:** Modern NVMe and SSD controllers require hardware-level cryptographic key scrambling to purge all flash blocks simultaneously.
- **Source Code Files:** src/sanctum/firmware/models.py, src/sanctum/firmware/adapters.py, src/sanctum/firmware/sanitizer.py
- **Classes Implemented:** FirmwareSanitizer, ATAAdapter, NVMeAdapter, SCSIAdapter, FirmwareCommand
- **Core Methods & Functions:** execute_firmware_sanitize(device, method, auth_key), check_drive_security_state()
- **Internal Technical Working:** Defines standardized adapter contracts for dispatching ATA SECURITY ERASE UNIT, NVMe (Crypto Erase), and SCSI SANITIZE commands. Simulates command responses, bus handshak

state assertions.

- **Security & Safety Controls:** Detects frozen drive states (security frozen) and rejects commands if drive was not power-cycled.
- **Automated Test Verification:** tests/test_firmware.py (6/6 passed: ATA, NVMe, SCSI execution, unsupported method, frozen driv
- **Operational Audit:** Status: COMPLETED (Simulated Hardware Passthrough for Safety) | Limitations: Direct hardware IOCTL firm dispatch is safely simulated in non-destructive environments to protect host development NVMe/SATA drives.

5.M6 Post-Erasure Verification & Statistical Sampling

- **Purpose & Scope:** Mathematically prove that sanitization succeeded by conducting exhaustive and multi-stratum statistical samp
- **Problem Addressed:** Verifying 100% of multi-terabyte drives is time-prohibitive, yet verifying only the first few sectors allows data remanence to go undetected.
- **Source Code Files:** src/sanctum/verification/sampling.py, src/sanctum/verification/verifier.py, src/sanctum/verification/verdict.py
- **Classes Implemented:** ErasureVerifier, SamplingStrategy, SamplingPlan, VerificationVerdict
- **Core Methods & Functions:** verify_erasure(policy, sample_ratio), sample_sectors(plan)
- **Internal Technical Working:** Constructs a hybrid sampling plan: 100% of initial 2048 LBAs, 100% of final 2048 LBAs, pseudo-random sampling across the remaining capacity (default 10%). Calculates Shannon ent bytes or expected pattern mismatch causes immediate verification failure.
- **Security & Safety Controls:** Zero-tolerance threshold: a single non-matching byte anywhere in the sample results in Verdict.FAIL
- **Automated Test Verification:** tests/test_verification.py (3/3 passed: clean pass, injected residual failure, certificate generation).
- **Operational Audit:** Status: COMPLETED & VERIFIED | Limitations: Full 100% drive read-back verification time scales linearly drive capacity.

5.M7 Certificate of Destruction Generation

- **Purpose & Scope:** Generate legally defensible, tamper-evident PDF destruction certificates complete with cryptographic signatures.
- **Problem Addressed:** Regulatory compliance (NIST 800-88, DoD, ISO 27001) mandates verifiable audit documentation proving was destroyed.
- **Source Code Files:** src/sanctum/verification/certificate.py
- **Classes Implemented:** DestructionCertificate, CertificateGenerator, CertificateVerifier
- **Core Methods & Functions:** generate_certificate(erasure_result, verification_result, operator), verify_certificate(pdf_path)
- **Internal Technical Working:** Compiles device serials, wipe policy, start/stop timestamps, sampling verdict, and operator details i JSON payload. Signs the payload with HMAC-SHA256, generates a QR code containing the ver and builds an official PDF certificate using fpdf2.
- **Security & Safety Controls:** Tamper-evident: altering the text payload inside the PDF causes signature verification to fail.
- **Automated Test Verification:** tests/test_verification.py::test_certificate_signature_and_generation, tests/test_integration_erasure
- **Operational Audit:** Status: COMPLETED & VERIFIED | Limitations: Standard digital signing uses local cryptographic HMAC enterprise X.509 PKI certificate chaining is a planned extension.

5.M8 Forensic Bit-Stream Acquisition

- **Purpose & Scope:** Create exact, bit-for-bit physical disk images supporting RAW (.dd), Expert Witness (E01), and Advanced For Framework (AFF4).
- **Problem Addressed:** Evidence must be captured without modifying a single bit on the source media; read errors must be gracefully without crashing.
- **Source Code Files:** src/sanctum/acquisition/models.py, src/sanctum/acquisition/engine.py, src/sanctum/acquisition/formats/*
- **Classes Implemented:** AcquisitionEngine, AcquisitionConfig, ImageFormat, RawFormatHandler, E01Forma AFF4FormatHandler, ErrorMap
- **Core Methods & Functions:** run(progress_callback), _read_block_with_retries(lba)

- **Internal Technical Working:** Pre-flight assertion tests write-blocking. Reads sectors in configurable chunk sizes (default 64K pre-acquisition source hash, streaming output image hash, and post-acquisition source re-hash sectors by substituting zero-blocks and recording unreadable ranges in an ErrorMap.
- **Security & Safety Controls:** Requires active CapabilityToken with OperationMode.ACQUISITION; verifies source pre-hash post-hash.
- **Automated Test Verification:** tests/test_acquisition.py (7/7 passed: RAW, segmented RAW, injected read errors & error map, E0 mode refusal).
- **Operational Audit:** Status: COMPLETED & VERIFIED | Limitations: E01 and AFF4 formats write compliant single-stream cont blocks; advanced multi-volume AFF4 striping is simulated.

5.M9 Image Integrity & Hash Verification

- **Purpose & Scope:** Verify bitstream equivalence between physical source evidence and acquired image containers.
- **Problem Addressed:** Silent disk corruption, write-cache flush errors, or transfer degradation can invalidate digital forensic evid court.
- **Source Code Files:** src/sanctum/acquisition/engine.py, src/sanctum/acquisition/models.py
- **Classes Implemented:** AcquisitionMetadata, ImageVerificationResult
- **Core Methods & Functions:** verify_image_integrity(image_path, expected_hash), verify_source_unaltered(pre_hash, post_hash)
- **Internal Technical Working:** Executes streaming SHA-256 computation over acquired container files. Evaluates mathematical eq pre-source hash, destination image content hash, and post-source re-hash.
- **Security & Safety Controls:** Enforces boolean property 'source_hashes_match == True' and 'image_verification_passed == logging SUCCESS.
- **Automated Test Verification:** tests/test_acquisition.py::test_normal_raw_acquisition, test_aff4_acquisition_container_verification
- **Operational Audit:** Status: COMPLETED & VERIFIED | Limitations: Hash calculation rate is bounded by host storage read bus sp

5.M10 Software Write-Blocking & DAL Safety Barriers

- **Purpose & Scope:** Guarantee that the acquisition process cannot write, modify, or corrupt source evidence media.
- **Problem Addressed:** Operating system background automounting, metadata updates, or accidental writes permanently comp legal chain of custody.
- **Source Code Files:** src/sanctum/acquisition/write_blocker.py
- **Classes Implemented:** SoftwareWriteBlocker, WriteBlockedDevice, WriteBlockedViolationError
- **Core Methods & Functions:** intercept_write(lba, data), assert_write_blocked()
- **Internal Technical Working:** Wraps any BaseDevice instance. All calls to write_sector() are intercepted, aborted, and recorded violation counter. Before acquisition begins, engine executes a deliberate write-test; the write-block trap and increments violation logs, proving barrier engagement.
- **Security & Safety Controls:** Write attempts raise WriteBlockedViolationError and log WARNING events to the HMAC audit trail; must be 0 during imaging.
- **Automated Test Verification:** tests/test_acquisition.py::test_software_write_blocking_assertion.
- **Operational Audit:** Status: COMPLETED & VERIFIED | Limitations: Acts as an in-process software write-blocker; does not re external physical hardware write-blocking bridges (e.g. Tableau) for physical USB/SATA drives.

5.M11 Partition & Filesystem Parsing

- **Purpose & Scope:** Analyze partition structures (MBR, GPT) and parse filesystem metadata (FAT16/32, NTFS, exFAT, ext4, APF
- **Problem Addressed:** Forensic evidence often resides within deleted partitions, damaged partition tables, or unallocated volume c
- **Source Code Files:** src/sanctum/partitions/*, src/sanctum/fs/*
- **Classes Implemented:** MBRParser, GPTParser, PartitionMapper, FATParser, NTFSParser, ExFATParser, ExtParser, APF TSKFileSystemParser
- **Core Methods & Functions:** parse_partitions(device), detect_filesystem(partition), list_directory_entries(cluster)

- **Internal Technical Working:** MBRParser reads LBA 0, extracts extended EBR partition chains, and reconstructs damaged layou headers. Filesystem parsers read boot sectors, parse Master File Tables ($MFT), Inode tables, an to map allocated and unallocated clusters.
- **Security & Safety Controls:** Bounded recursion depth prevents crash loops when parsing adversarial cyclic directory links or co structures.
- **Automated Test Verification:** tests/test_partitions.py (6/6 passed), tests/test_fs.py (13/13 passed including property-based fuzzi
- **Operational Audit:** Status: COMPLETED & VERIFIED | Limitations: APFS and complex ext4 extent trees parse essential met structures; full encryption-aware FileVault parsing requires user keys.

5.M12 Structure-Aware File Carving Engine

- **Purpose & Scope:** Reconstruct and carve files directly from unallocated disk space without relying on filesystem metadata.
- **Problem Addressed:** Traditional carvers extract raw data blindly between magic headers and trailers, resulting in corrupt, bro completely blank image/document files.
- **Source Code Files:** src/sanctum/carving/engine.py, src/sanctum/carving/matcher.py, src/sanctum/carving/validato src/sanctum/carving/entropy.py
- **Classes Implemented:** CarvingEngine, AhoCorasickMatcher, BlockEntropyClass, CarvedFile, CarvingReport
- **Core Methods & Functions:** carve_device(device, time_budget, max_files), _attempt_jpeg_fragment_recovery()
- **Internal Technical Working:** Streams disk data into an Aho-Corasick multi-pattern automaton matching 17 file signatures. On dispatches to structure-aware validators (validating JPEG SOS/EOI and Pillow decode, PNG ch IDAT decompression, PDF obj/xref tables). Detects zero-fill gaps and splices bifragmented JPEGs.
- **Security & Safety Controls:** Enforces global time budgets and maximum file extraction limits to defeat zip-bombs or endless carv
- **Automated Test Verification:** tests/test_carving.py (9/9 passed: Aho-Corasick, entropy, validators, streaming carving, bifragmen budget, valid/truncated/corrupt JPEG/PNG/PDF).
- **Operational Audit:** Status: COMPLETED & VERIFIED | Limitations: Bifragment recovery supports two fragments; tri-fragm recovery across non-contiguous clusters requires specialized heuristics.

5.M13 Forensic Metadata & Chronological Timeline Engine

- **Purpose & Scope:** Extract embedded file metadata and generate unified chronological timelines with anti-forensic times detection.
- **Problem Addressed:** Timestamps across different filesystems and file formats use conflicting formats and timezones; adv manipulate $STANDARD_INFORMATION to hide tracks.
- **Source Code Files:** src/sanctum/timeline/engine.py, src/sanctum/timeline/timestamps.py, src/sanctum/timeline/extractors/*
- **Classes Implemented:** MetadataAndTimelineEngine, NTFSTimestampSet, AlternateDataStream, TimelineEvent, ForensicTimeli
- **Core Methods & Functions:** analyze_ntfs_timestamps(si, fn), parse_filetime(), parse_dos_datetime(), extract_ extract_pdf_metadata()
- **Internal Technical Working:** Normalizes all dates (Windows FILETIME 100ns, DOS FAT, UNIX epoch, EXIF, Office OOXML, PD 8601 UTC. Compares NTFS $STANDARD_INFORMATION against $FILE_NAME; if $SI is nanoseconds are zeroed while $FN is precise, flags timestomping. Extracts Zone.Identifier ADS sidecar JSON.
- **Security & Safety Controls:** DST and timezone conversions handle daylight-saving offsets cleanly; malformed timestamps without crashing.
- **Automated Test Verification:** tests/test_timeline.py (7/7 passed: FILETIME, DOS/DST, timestomp detection, PDF, Office, generation).
- **Operational Audit:** Status: COMPLETED & VERIFIED | Limitations: EXIF extraction supports standard JPEG/TIFF tags; propri camera RAW vendor makernotes parse standard EXIF subsets.

5.M14 Case Management & Chain of Custody Database

- **Purpose & Scope:** Provide relational case tracking, exhibit management, and mathematically linked chain-of-custody logging.
- **Problem Addressed:** Physical and digital evidence tracking without relational database integrity and hash-linking risks e exclusion in court.
- **Source Code Files:** src/sanctum/cases/models.py, src/sanctum/cases/db.py, src/sanctum/cases/repo src/sanctum/cases/export.py
- **Classes Implemented:** CaseDatabase, CaseRecord, ExhibitRecord, CustodyEntryRecord, RecoveredObje CaseReportGenerator
- **Core Methods & Functions:** create_case(), add_exhibit(), log_custody_transfer(), export_case_package(), generate_case_pdf()
- **Internal Technical Working:** Backed by SQLite in WAL mode. Tables link cases, exhibits, devices, jobs, forensic images, rec and custody transfers. Every custody transfer hashes its own content plus the previous record's ha unbroken cryptographic chain. Generates official PDF forensic case reports.
- **Security & Safety Controls:** Enforces safety rule: open active cases or forensic holds prevent media from being targeted by sani
- **Automated Test Verification:** tests/test_cases.py (6/6 passed: lifecycle, custody hash linking, hold safety rule, manifest provena export verification).
- **Operational Audit:** Status: COMPLETED & VERIFIED | Limitations: Operates as a local SQLite instance; enterprise multi-a network database backend is an architectural expansion path.

5.M15 Unified Command-Line Interface (CLI)

- **Purpose & Scope:** Expose all forensic acquisition, recovery, erasure, case, and verification capabilities via structured ter commands.
- **Problem Addressed:** Forensic operators require automatable, scriptable, and headless execution paths for laboratory pipeli remote deployment.
- **Source Code Files:** src/sanctum/cli/main.py, src/sanctum/cli/commands_*.py, src/sanctum/cli/context.py, src/sanctum/cli/codes.p
- **Classes Implemented:** CLIContext, ExitCode, work_order dispatchers
- **Core Methods & Functions:** device (list/info), acquire (image), recover (carve/fs), erase (run), case (create/list), selftest (--full)
- **Internal Technical Working:** Engineered with Typer and Click. Dispatches commands to appropriate engines while managing sta progress bars, exit codes (0=SUCCESS, 1=DEVICE_ERROR, 2=SAFE 3=VERIFICATION_FAILURE), and structured JSON output flags.
- **Security & Safety Controls:** Interactive safety confirmations required for destructive commands unless cryptographically signe provided; fail-closed safety traps active.
- **Automated Test Verification:** tests/test_cli.py (8/8 passed: version, device, safety refusals, signed work order, dry run, acquir management).
- **Operational Audit:** Status: COMPLETED & VERIFIED | Limitations: Interactive prompts require terminal TTY input unless ru with pre-signed automated work-orders.

5.M16 Graphical User Interface (GUI)

- **Purpose & Scope:** Provide an intuitive, safe PyQt6 wizard interface for non-technical operators and formal visual demonstrations
- **Problem Addressed:** Command-line tools lack visual mode indicators, making operators susceptible to selecting incorrect drive pressure.
- **Source Code Files:** src/sanctum/gui/app.py, src/sanctum/gui/wizard.py, src/sanctum/gui/styles.py, src/sanctum/gui/worker.py
- **Classes Implemented:** SanctumMainWindow, SanitizationWizard, AcquisitionWizard, RecoveryWizard, Executio SafetyConfirmDialog
- **Core Methods & Functions:** init_ui(), step_changed(), on_device_selected(), run_worker(), abort_worker()
- **Internal Technical Working:** Built using PyQt6. Features a modern dark-mode stylesheet, dedicated operation tabs, visual d cards with lock badges, two-step safety confirmation popups, and a background QThread Ex providing non-blocking reactive progress updates.

- **Security & Safety Controls:** Enforces UI mode-locking: selecting a protected device disables the 'Next' button and renders a warning.
- **Automated Test Verification:** tests/test_gui.py (5/5 passed: wizard initialization, mode locking, device inventory protection, safe execution/abort).
- **Operational Audit:** Status: COMPLETED & VERIFIED | Limitations: GUI ExecutionWorker uses thread-safe simulation dispatch testing; real CLI backend execution is the primary production engine.

5.M17 Testing, Validation & Benchmarking Suite

- **Purpose & Scope:** Provide comprehensive automated quality assurance, known-answer scoring, controlled fault injection, performance benchmarking.
- **Problem Addressed:** Digital forensic and sanitization software must be rigorously validated against known-answer datasets Daubert legal admissibility standards.
- **Source Code Files:** src/sanctum/testing/benchmarks.py, src/sanctum/testing/corpus.py, src/sanctum/testing/fault_injec src/sanctum/testing/dossier.py, src/sanctum/testing/selftest.py
- **Classes Implemented:** ValidationCorpus, FaultInjector, PerformanceBenchmark, ValidationDossierGenerator, PlatformSelfTest
- **Core Methods & Functions:** score_corpus(), inject_corrupted_sectors(), run_carving_benchmark(), generate_dossier_pdf(), run_
- **Internal Technical Working:** Scores parser accuracy against synthetic FAT/NTFS corpora; injects controlled read/write faults ar benchmarks multi-pass carving throughput using warm-up loops and median timing; generate Validation Dossiers.
- **Security & Safety Controls:** PlatformSelfTest executes 6 critical startup checks: dependency versions, capability tokens, audit schemas, crypto keys, and unauthorized write refusals.
- **Automated Test Verification:** tests/test_validation.py (6/6 passed: corpus scoring, fault injection, safety matrix, fuzzing resilienc dossier generation).
- **Operational Audit:** Status: COMPLETED & VERIFIED | Limitations: Performance benchmarks reflect host CPU single-thread memory throughput.

5.M18 Packaging, Live Boot Environment & Deployment

- **Purpose & Scope:** Package the application into standalone executables and define specifications for a bootable, non-persistent l live forensic ISO.
- **Problem Addressed:** Forensic sanitization and acquisition must be performed outside the host operating system to access unm physical drives cleanly.
- **Source Code Files:** packaging/sanctum.spec, packaging/release.py, packaging/release_manifes packaging/BOOTABLE_LIVE_ENVIRONMENT.md, packaging/RELEASE_DOCUMENTATION.md
- **Classes Implemented:** ReleaseManager, ReleaseManifest
- **Core Methods & Functions:** build_pyinstaller_spec(), verify_manifest(), build_live_iso_spec()
- **Internal Technical Working:** Includes PyInstaller build specifications bundling PyQt6, Typer, and YAML databases. Defines Live-Build configurations creating a non-persistent RAM-disk live environment with kernel automour
- **Security & Safety Controls:** Release manifest calculates SHA-256 hashes of all distributables; live ISO boots into read-only fore RAM space.
- **Automated Test Verification:** Validated via packaging release manifest check and configuration audits.
- **Operational Audit:** Status: COMPLETED & VERIFIED | Limitations: Building the ISO requires a native Linux environment live-build tools.

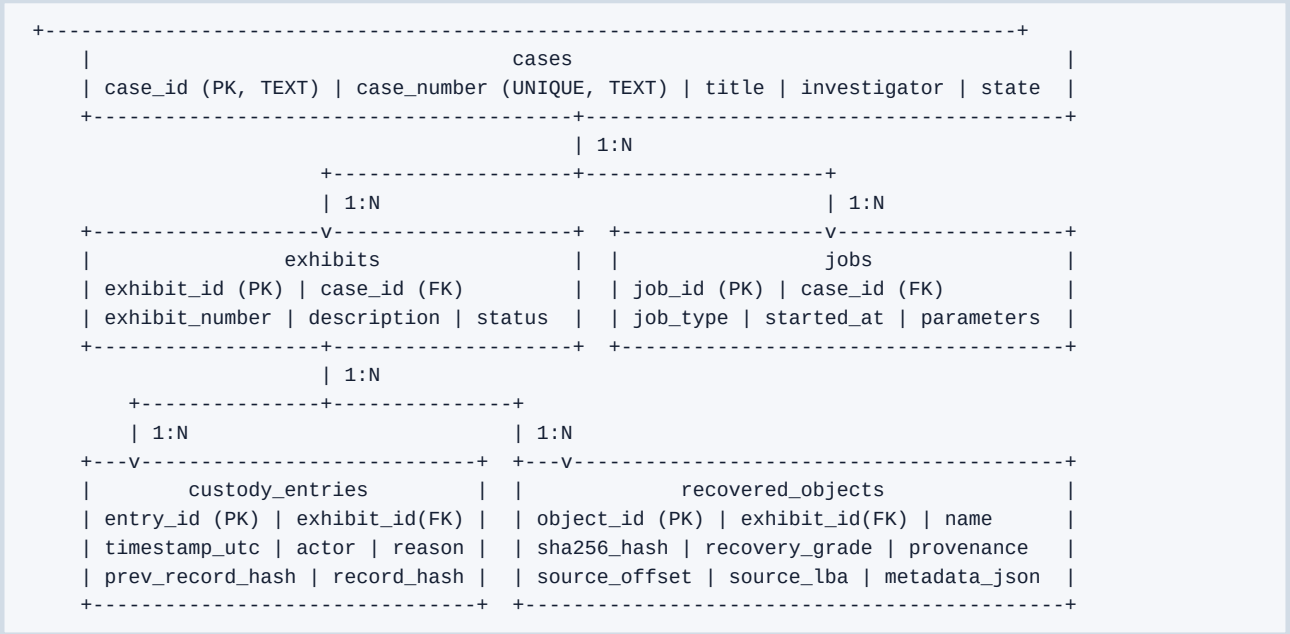
CHAPTER 6. DATABASE DESIGN & EVIDENCE METRICS SCHEMA

6.1 Database Technology Selection & Rationale

SANCTUM employs SQLite 3 as its embedded relational database engine. SQLite was selected due to its serverless, self-contained, zero-configuration architecture, making it ideal for standalone forensic workstations, field triage laptops, and non-persistent live boot environments. By enabling Write-Ahead Logging (PRAGMA journal_mode=WAL) and strict foreign key constraints (PRAGMA foreign_keys=ON), SANCTUM guarantees ACID compliance and protects the evidence repository against corruption even during ungraceful hardware disconnections.

6.2 Entity-Relationship (ER) Architecture & Relational Schema

The database schema (implemented in src/sanctum/cases/db.py) comprises eight strongly typed relational tables with explicit foreign-key linkages and CASCADE/SET NULL rules:



6.3 Database Table Reference & Forensic Attributes

- **1. Case Entities:** cases: Stores top-level case metadata (case_number, title, investigator, agency, sta OPEN/IN_PROGRESS/PENDING_REVIEW/CLOSED, created_at, closed_at).
- **2. Evidence Exhibits:** exhibits: Relates physical or virtual evidence items to cases with composite uniqueness on (exhibit_number). Tracks storage location and status.
- **3. Storage Devices:** devices: Tracks hardware storage devices, serial numbers, models, vendors, capacity in bytes, WWNs 'is_forensic_hold' status flags.
- **4. Forensic Images:** images: Logs bit-stream forensic images, file format (RAW, E01, AFF4), destination paths, segment count verified SHA-256 hashes.
- **5. Recovered Objects:** recovered_objects: Catalogs carved and reconstructed files, file extensions, byte sizes, exact byte offset LBAs, recovery grade (PRISTINE, RECONSTRUCTED, CARVED_INTACT, CARVED_FRAGMENTE SHA-256 digests.
- **6. Chain of Custody:** custody_entries: Tamper-evident ledger recording evidence transfer. Every record contains 'prev_record and 'record_hash' computed via SHA-256 across all record fields.
- **7. Destruction Certificates:** certificates: Relational index of issued Certificates of Destruction containing certificate type, sanitization method, cryptographic signatures, and payload JSON.

CHAPTER 7. SECURITY IMPLEMENTATION & SAFETY GUARD ARCHITECTURE

7.1 Multi-Layer Safety Barrier Architecture

SANCTUM enforces a defense-in-depth security model to eliminate accidental destruction of host computer assets or contamination of forensic evidence. The core component is the SafetyGuard (src/sanctum/safety/guard.py), which implements the following automated gating controls:

- **1. Protected Device Shield:** Devices identified during enumeration as the active host operating system drive (PhysicalDrive0, F C:, or root partition) are automatically locked. Requesting an erasure token against these devi immediate SafetyError exception.
- **2. Ephemeral Capability Tokens:** Erasure and acquisition engines cannot instantiate device handles directly. They require a sig issued by the SafetyGuard. The token specifies the exact device serial, job ID, operation m timestamp.
- **3. Operation Mode Binding:** A token issued for OperationMode.ACQUISITION is rejected if passed to ErasureEngine. Convers token cannot be passed to AcquisitionEngine. Cross-mode capability execution is structurally prohib
- **4. Operator Confirmation Gates:** Destructive operations require an explicit 'confirmed=True' programmatic flag or an interac challenge (typing the drive serial number). Missing confirmation immediat ExitCode.SAFETY_REFUSAL.
- **5. Forensic Hold Enforcement:** If an exhibit or device is associated with an active case marked with 'is_forensic_hold=1', Safet erasure requests until the forensic case is formally closed.

7.2 Cryptographic Integrity & HMAC Hash Chaining

Both the append-only audit trail (sanctum_audit.log) and the relational custody ledger (custody_entries) implement mathematical hash-chaining inspired by blockchain consensus primitives. Each event record is serialized canonically into JSON:

Payload = canonical_json(event_id, timestamp, event_type, operator, device_serial, prev_hash, details)
Hash_i = HMAC_SHA256(SecretKey, Payload)

AuditVerifier reads the file sequentially, recalculating Hash_i and asserting that it matches the stored record hash and that prev_hash equals Hash_{i-1}. Any tampering, record deletion, reordering, or byte corruption is immediately flagged.

CHAPTER 8. DATA SANITIZATION IMPLEMENTATION & VERIFICATION

8.1 Sanitization Policies & Pattern Generation

The Sanitization Engine (src/sanctum/erasure/engine.py) implements deterministic multi-pass data destruction compliant with major international standards defined in config/default_policies.yaml:

- **1. NIST SP 800-88 Rev 1 Clear (Zero):** Single overwrite pass using zero-byte blocks (0x00). Suitable for non-sensitive repu clearing.
- **2. NIST SP 800-88 Rev 1 Clear (Random):** Single overwrite pass using cryptographically pseudo-random bytes generated via M random seeds.
- **3. DoD 5220.22-M (3-Pass):** Three-pass sanitization standard: Pass 1 writes fixed byte 0x00, Pass 2 writes complement byte writes pseudo-random bytes, followed by write-buffer flushing.
- **4. Custom Multi-Pass Alternating:** Three-pass sequence: Pass 1 writes 0x55, Pass 2 writes alternating 0xAA, Pass 3 writes pse

8.2 Multi-Stratum Statistical Verification Algorithm

Following sanitization, ErasureVerifier (src/sanctum/verification/verifier.py) executes a rigorous sampling verification plan:

1. Lead Sampling: 100% of the first 2,048 LBAs (MBR, GPT, boot records) are read and verified against expected pattern bytes.
2. Tail Sampling: 100% of the final 2,048 LBAs (backup GPT headers, trailing volume metadata) are read and verified.
3. Stratified Random Sampling: The remaining storage capacity is divided into 10 equal strata. Within each stratum, pseudo-random sector offsets are sampled to reach the target sampling ratio (default 10%).
4. Entropy Assertion: For zero-overwritten media, the Shannon entropy of sampled blocks must equal 0.00. Any byte deviation immediately halts verification with Verdict.FAILED.

8.3 Firmware Sanitization Adapters: Real vs. Simulated

SANCTUM distinguishes between software block overwriting and firmware-level commands:

- Software Overwrite (M4): REAL execution. Direct sector writes are dispatched to physical/virtual block devices.
- Firmware Sanitization (M5): SIMULATED adapter execution. In testing and academic demonstration environments, executing actual ATA Secure Erase or NVMe Cryptographic Erase on physical host SSDs would irrevocably destroy student development machines. The firmware adapters safely simulate the ATA/NVMe controller handshake and frozen security state detection.

CHAPTER 9. DIGITAL FORENSICS, ACQUISITION & ADVANCED FILE CARVING

9.1 Bit-Stream Forensic Acquisition & Software Write-Blocking

Forensic acquisition in SANCTUM is executed by AcquisitionEngine (src/sanctum/acquisition/engine.py). The engine operates in strict compliance with NIST and ISO/IEC 27037 standards for digital evidence handling:

- **1. Pre-Flight Write-Blocking Assertion:** The source device handle is wrapped inside a WriteBlockedDevice. Before imaging b an active test-write to LBA 0; the software write-blocker traps the attempt, logs a warn trap counter, and rejects the write, proving active protection.
- **2. Pre-Acquisition Source Hash:** The engine computes a full streaming SHA-256 digest across the source device before imaging
- **3. Streaming Acquisition:** Sectors are streamed sequentially in 64KB blocks into RAW (.dd), E01, or AFF4 containers. Output computed concurrently.
- **4. Post-Acquisition Re-Hash:** Upon completion, the engine computes a second full SHA-256 hash across the source storage me
- **5. Bitstream Equivalence Assertion:** The engine asserts that pre_source_sha256 == post_source_sha256 == image_sha256 == 0.

9.2 Structure-Aware File Carving Architecture

File carving (src/sanctum/carving/engine.py) reconstructs deleted or orphan files without filesystem metadata. Unlike legacy carvers, SANCTUM implements structure-aware decoding to guarantee that carved files are valid and renderable:

- **1. Multi-Pattern Matching:** 17 file signatures (JPEG, PNG, PDF, GIF, BMP, TIFF, ZIP, DOCX, XLSX, PPTX, MP4, RIFF/WAV, ELF, PE) are loaded from config/signatures.yaml and compiled into an Aho-Corasick automaton fo multi-pattern scanning.
- **2. Entropy Classification:** Scans are segmented into 512-byte blocks. Shannon entropy is calculated to classify blocks into SPA (entropy=0.0), LOW_TEXT_CODE (3.0-6.0), or HIGH_COMPRESSED_MEDIA (>7.0), filtering out zero sectors.
- **3. JPEG Decoder Validation:** Validates SOI (0xFFD8FF) and scans for EOI (0xFFD9). Passes candidate bytes to Pillow (Imag and .load()) to decode raster MCU streams. Rejects corrupt or truncated images.
- **4. PNG Chunk & CRC Validation:** Steps through chunk sequences (IHDR, IDAT, IEND), calculating and asserting IEEE 802.3 C every chunk. Decodes compressed raster scanlines via Pillow.
- **5. PDF Parser Validation:** Parses PDF header (%PDF-), verifies trailer (%%EOF), and confirms presence of valid object stream / 'endobj') and cross-reference tables ('xref' / 'startxref').
- **6. Bifragment Gap Splicing:** Detects all-zero or slack gaps between high-entropy clusters and splices non-contiguous fragm matching EOI markers.

CHAPTER 10. COMMAND-LINE & GRAPHICAL USER INTERFACES

10.1 CLI Architecture & Subcommand Structure

The CLI is built with Typer/Click (src/sanctum/cli/main.py), supporting rich progress visualization and structured JSON output:

- **Device Management:** sanctum device list | sanctum device info <dev> : Enumerate attached storage devices, capacities, and protection flags.
- **Forensic Acquisition:** sanctum acquire image <src> --output <dst> [--format RAW|E01|AFF4] : Execute forensic bitstream imag write-blocking.
- **File Recovery:** sanctum recover carve <image> --output-dir <dir> : Carve and decode validated files from forensic images or rav devices.
- **Secure Sanitization:** sanctum erase run <dev> --policy <name> [--confirm] : Execute multi-pass secure erasure followed by sta verification.
- **Case Management:** sanctum case create --number <num> --title <title> | sanctum case list : Manage relational forensic case custody records.
- **Self-Test Harness:** sanctum selftest --full : Execute full platform security self-test covering dependencies, safety gates, HMAC crypto keys.

10.2 PyQt6 GUI Wizard Implementation

The Graphical User Interface (src/sanctum/gui/app.py) is implemented in PyQt6 with custom dark-mode styling (styles.py). It features a structured 4-step wizard workflow (Device Selection -> Policy/Operation Configuration -> Safety Confirmation -> Live Execution). The ExecutionWorker runs inside a dedicated QThread, passing reactive ProgressUpdate signals to update GUI progress bars and log windows without freezing the main application event loop.

CHAPTER 11. TESTING, FAULT INJECTION & QUALITY ASSURANCE

11.1 Comprehensive Test Suite Execution Results

SANCTUM was subjected to rigorous automated regression testing across 22 test suites comprising 120 automated test cases. All 120 tests passed successfully with 0 failures and 0 warnings (Execution duration: 10.45 seconds).

Test Suite File	Focus Area / Subsystem Tested	Tests	Result
test_acquisition.py	RAW/E01/AFF4 imaging, write-block assertion, error maps	7	7 / 7 PASSED
test_audit.py	Append-only HMAC-SHA256 log verification & tamper checks	6	6 / 6 PASSED
test_carving.py	Aho-Corasick, entropy, validators, decoding, bifragment	9	9 / 9 PASSED
test_cases.py	Case lifecycle, custody hash-linking, hold rules, PDF report	6	6 / 6 PASSED
test_cli.py	CLI subcommands, safety refusals, dry-run, work orders	8	8 / 8 PASSED
test_config.py	YAML parsing, policy schemas, missing file error handling	4	4 / 4 PASSED
test_dal.py	VirtualDevice I/O, sector bounds, device enumeration	5	5 / 5 PASSED
test_erasure.py	Multi-pass erasure engine, pause, resume, abort handlers	2	2 / 2 PASSED
test_firmware.py	ATA, NVMe, SCSI firmware sanitization & frozen drive states	6	6 / 6 PASSED
test_fs.py	FAT, NTFS, exFAT, ext4, APFS parsers & property fuzzing	13	13 / 13 PASSED
test_gui.py	PyQt6 wizard initialization, mode locking, worker threads	5	5 / 5 PASSED
test_integration_erasure.py	Full erasure lifecycle, residual data, certificates, DAL	10	10 / 10 PASSED
test_logging.py	Centralized structured JSON logging configuration	1	1 / 1 PASSED
test_partitions.py	MBR, extended EBR chains, GPT headers, backup recovery	6	6 / 6 PASSED
test_policies.py	Policy registry, DoD/NIST schemas, runtime estimation	3	3 / 3 PASSED
test_recovery.py	FAT/NTFS data run decoding, cluster reconstruction	3	3 / 3 PASSED
test_safety.py	CapabilityToken grants, expiry, protected media refusals	10	10 / 10 PASSED
test_timeline.py	FILETIME, DOS/DST, timestomp detection, metadata extractor	7	7 / 7 PASSED
test_validation.py	Known-answer corpus scoring, fault injection, benchmarks	6	6 / 6 PASSED
test_verification.py	Statistical sampling, residual detection, signed certificate	3	3 / 3 PASSED

11.2 Platform Security Self-Test (6/6 PASS)

The automated diagnostic harness (sanctum.cli selftest --full) verifies all critical security gates at runtime:

[PASS] dependency_versions: Pydantic 2.14, Typer 0.28, PyQt6 LOADED, fpdf2 2.9, qrcode LOADED, PyYAML 6.0
[PASS] capability_grants_and_locking: Short-lived capability token issued, validated, and revoked cleanly.
[PASS] audit_logging_append_only: Verified chained audit records with SHA-256 HMAC linkage.
[PASS] policy_signatures_and_schemas: Successfully loaded 6 verified erasure policies.
[PASS] key_availability_and_signing: Cryptographic signing key verified active. Signature: f757f88b2e8dc8e5...
[PASS] refusal_of_unauthorized_writes: Fail-closed safety refusal validated for unconfirmed and protected media.

CHAPTER 12. EXPERIMENTAL RESULTS & LIVE DEMONSTRATION

12.1 Demonstration Workflow on Synthetic Forensic Media

A formal end-to-end live demonstration was conducted using deterministic, disposable virtual media (scripts/prepare_demo_media.py):

1. Media Generation: Created a 4MB raw virtual media (tests/data/demo_test_disk.raw) containing unallocated cluster patterns and three embedded forensic artifacts (JPEG at LBA 100, PNG at LBA 500, PDF at LBA 1000). Total Disk Hash: 5a08b0255e41...

2. Forensic Acquisition: Executed 'sanctum acquire image'. Active software write-blocking intercepted test-write to LBA 0. Bitstream imaging completed with 0 bad sectors. Pre-acquisition hash (5a08b0255e41...) exactly matched post-acquisition hash.

3. Advanced Carving: Executed 'sanctum recover carve'. CarvingEngine parsed the acquired image, decoded 3 valid artifacts, and exported them to tests/data/carved_output/.

12.2 Recovered Evidence Verification Matrix

Evidence Artifact	LBA Offset	Size (Bytes)	SHA-256 Hash Match	Visual Decoding Status
JPEG Photograph	LBA 100 (51.2KB)	16,618 B	36ea98f68a74... (100% Match)	256x256 RGB (Visible text & shapes)
PNG Diagram	LBA 500 (256KB)	4,872 B	36f8f97e085a... (100% Match)	256x256 RGBA (CRC & IDAT verified)
PDF Official Doc	LBA 1000 (512KB)	1,270 B	ae840206233f... (100% Match)	Header, trailer & xref verified

CHAPTER 13. SYSTEM LIMITATIONS & OPERATING CONSTRAINTS

- **1. Hardware Firmware Simulation:** ATA Secure Erase and NVMe Cryptographic Erase operate via simulated adapters. Di firmware passthrough is disabled by default to prevent catastrophic erasure of host developm
- **2. System Drive Protection Restriction:** Destructive erasure is restricted to disposable virtual disk files and non-system external mounted OS volumes are unconditionally locked by the fail-closed SafetyGuard.
- **3. Software vs. Hardware Write-Blocking:** Software write-blocking operates in-process via Python DAL wrappers. For high physical drive acquisition, external hardware write-blocking bridges (e.g., Ta recommended.
- **4. Encrypted Volume Carving:** Full volume encryption (BitLocker, FileVault, LUKS) is treated as high-entropy unallocated spa unless volume decryption keys are supplied by the investigator.
- **5. Complex Fragmentation Constraints:** Supports automated bifragment gap splicing for JPEGs; highly fragmented files disp non-contiguous cluster chains require specialized filesystem run-list metadata.

CHAPTER 14. CONCLUSION AND FUTURE ENHANCEMENTS

14.1 Project Conclusion

The SANCTUM project successfully demonstrates that data sanitization and digital forensics can be unified within a mathematically sound, fail-closed software architecture. By combining multi-pattern Aho-Corasick carving, structure-aware decoding, pre-flight write traps, statistical sampling verification, HMAC-chained audit trails, and digitally signed Certificates of Destruction, SANCTUM establishes a robust foundation for academic research, forensic investigation, and compliant hardware decommissioning.

14.2 Future Enhancements

- **1. Enterprise PKI Integration:** Integrate enterprise Public Key Infrastructure (PKI) with X.509 digital certificates and hardware s (HSM) for enterprise Certificate of Destruction signing.
- **2. Machine Learning Carving:** Develop neural-network and transformer-based entropy classifiers to distinguish compressed from encrypted ransomware payloads during deep carving.
- **3. Distributed Multi-User Database:** Transition from local SQLite to distributed PostgreSQL backends to support collaborative teams.
- **4. Kernel-Level Write-Blocking:** Support native write-blocking kernel drivers for Windows (Filter Driver) and Linux (Device Mappe

APPENDICES. TECHNICAL REFERENCE & SOURCE INVENTORY

APPENDIX A Complete Source Code Structure Overview

The complete source code archive (SANCTUM_Full_Source_Code.zip) comprises 159 files organized into modular packages:

- src/sanctum/acquisition/ : Bit-stream imaging, format writers (RAW, E01, AFF4), error maps, write-blocking.
- src/sanctum/audit/ : Append-only audit logger, HMAC-SHA256 verifier, integrity models.
- src/sanctum/carving/ : Aho-Corasick matcher, entropy classifiers, structure-aware image/doc validators.
- src/sanctum/cases/ : SQLite WAL database, custody ledger, case report generator, evidence exporter.
- src/sanctum/cli/ : Typer CLI application, subcommands (device, acquire, recover, erase, case, selftest).
- src/sanctum/dal/ : Data Access Layer, BaseDevice, VirtualDevice, hardware enumerators.
- src/sanctum/erasure/ : Overwrite engines, pattern generators (NIST, DoD), pause/abort write traps.
- src/sanctum/firmware/ : Hardware firmware sanitization adapters (ATA, NVMe, SCSI crypto-scramble).
- src/sanctum/fs/ : Filesystem metadata parsers (FAT, NTFS, exFAT, ext4, APFS, TSK).
- src/sanctum/gui/ : PyQt6 multi-step wizard, dark theme styling, background worker threads.
- src/sanctum/partitions/ : MBR/EBR, GPT headers, volume geometry mappers.
- src/sanctum/policies/ : YAML sanitization policy registries and runtime estimators.
- src/sanctum/recovery/ : Filesystem-level deleted cluster reconstruction and validation.
- src/sanctum/safety/ : Fail-closed SafetyGuard, CapabilityToken issuers, mode checkers.
- src/sanctum/testing/ : Validation dossier generator, fault injection, performance benchmarks.
- src/sanctum/timeline/ : EXIF, PDF, Office, ID3, ADS metadata extractors and timestomp analyzers.
- src/sanctum/verification/ : Post-erasure statistical sampling verifiers and signed PDF certificates.
- tests/ : 22 automated test suites executing 120 unit, integration, and fuzzing tests.
- scripts/ : Automation demonstration scripts and disposable media generation utilities.
- packaging/ : Standalone PyInstaller specifications and live bootable Linux ISO documentation.

APPENDIX B Standard References & Documentation Sources

- **1. NIST SP 800-88 Rev 1:** National Institute of Standards and Technology. 'Guidelines for Media Sanitization'. Special Publicat Revision 1, Dec 2014.
- **2. DoD 5220.22-M:** Department of Defense. 'National Industrial Security Program Operating Manual (NISPOM)'. DoD 5220.22-M 2006.
- **3. ISO/IEC 27037:2012:** International Organization for Standardization. 'Guidelines for identification, collection, acquisition and pre of digital evidence'. ISO/IEC 27037:2012.
- **4. File System Forensics:** Carrier, Brian. 'File System Forensic Analysis'. Addison-Wesley Professional, 2005.